

DCA: Discrete Computer Arithmetic - A Draft Oriented Toward Finite Computation

Author: Wang Bingqin Affiliation: Beijing National Accounting Institute Version: 2026-07-04

Abstract

This document abbreviates “Discrete Computer Arithmetic” as DCA. DCA here is not meant to replace existing mathematics, nor to claim a larger mathematical system. It is a modest, implementable, and checkable way to write mathematics for computation. This document cares about one thing: when objects eventually have to land in programs, chips, proof assistants, or finite data structures, how can we write definitions, operations, and arguments clearly enough?

The basic attitude of this document is: continuous objects may serve as external specifications, limiting intuition, or error-analysis tools, but the implementation layer directly handles only finite encodings, integers, rational numbers, finite graphs, finite matrices, finite-state machines, and finite proof objects. The benefit is that definitional boundaries become clear, arguments become easier to mechanically check, and code becomes easier to reproduce.

Common Conventions

1. Object convention: Every computable object in this document should have a finite encoding, denoted $\text{enc}(x) \in \{0,1\}^*$. Integers, finite lists, matrices, graphs, programs, and proof scripts all fall into this category.
2. Boundary convention: Every algorithm must state its word length, modulus, state space, loop bound, or termination condition. If an infinite set is used, it is used only as an external specification or generation rule, not as an object stored all at once.
3. Arithmetic convention: Addition at machine word length is, by default, wraparound arithmetic modulo 2^w . If saturation, truncation, arbitrary precision, or rational arithmetic is needed, it should be stated explicitly.
4. Proof convention: This document commonly uses four kinds of arguments: finite enumeration, structural induction, loop invariants, and simplification of algebraic identities. For propositions over finite fields or finite sets, exhaustive verification can also be a valid proof.
5. Approximation convention: When real numbers, complex numbers, differentiation, probability densities, quantum amplitudes, and similar concepts are involved, this document gives only discrete representations or integer-scaled representations, while retaining error bounds or applicability conditions.

Survey Materials

During the preparation of this document, several foundational sources were consulted: Turing’s paper on computability, Shannon’s communication theory, Cooley and Tukey’s FFT paper, materials on discrete exterior calculus and discrete differential geometry, Forman’s discrete Morse theory, Rocq/Coq documentation on inductive types, the RISC-V instruction-set documentation, papers on quantized integer neural networks, NIST post-quantum cryptography standards, and literature on the universality of cellular automata. They are listed in the references at the end. This document only draws from the parts related to finite representation, discrete structure, and implementable algorithms.

Existing Work and Examples

Further research confirms that many parts of this document are not speculative directions. They already appear, scattered across computational mathematics, formal verification, cryptography, optimization, graph algorithms, machine-learning deployment, operating systems, and hardware design. The following lists reference examples for the 43 parts. They are only “existing implementations or research leads” ; it does not mean that these projects use the name DCA.

1. Arithmetic Foundations: Mature implementations already exist for big integers and modular arithmetic, such as [GMP](#), [FLINT](#), and SageMath’s integer and finite-field interfaces. In cryptographic implementations, [Fiat-Crypto](#) generates finite-field arithmetic from Coq/Rocq proofs into C code; [HACL*](#) and EverCrypt bring high-assurance cryptographic primitives down to deployable C/assembly.
2. Algebraic Structures: [GAP](#) is oriented toward computational discrete algebra, especially computational group theory; [SageMath](#) integrates GAP, FLINT, NumPy, SciPy, Singular, and other systems into an open mathematical-computation environment; [arkworks algebra](#) implements finite fields, elliptic curves, and polynomials required by zkSNARKs in Rust.
3. Discrete Analysis: Finite-difference, finite-element, and finite-volume methods are already mainstream in scientific computing. [SciPy](#) and [PETSc](#) provide sparse matrices, difference-equation solving, and infrastructure for large-scale numerical computation; discrete Laplacians on graphs and meshes are used directly in libraries such as [libigl](#), [geometry-central](#), and GUDHI.
4. Discrete Geometry: [libigl](#) provides sparse discrete differential-geometry operators, cotangent Laplacians, and mesh-topology structures; [geometry-central](#) is centered on surface meshes and emphasizes using only the geometric data actually required by algorithms, such as edge lengths, intrinsic geometry, and adjacency structures.
5. Logic and Reasoning: SAT/SMT solvers are already foundational tools for software and hardware verification. [Kissat](#), [CaDiCaL](#), [Z3](#), and [cvc5](#) are directly reusable examples.
6. Discrete Fourier Analysis and NTT: Number-theoretic transforms are widely used in polynomial multiplication, zk proofs, and post-quantum cryptography. In 2025, IACR Communications in Cryptology included work on formally verifying core NTT operations; Winterfell, Plonky2, and arkworks also use similar structures in finite-field FFT/FRI/polynomial-commitment systems.
7. Discrete Probability: Finite sample spaces, counting probability, and discrete random variables already have many implementations in probabilistic programming and statistical software. [PyMC](#), [Stan](#), and [TensorFlow Probability](#) support discrete distributions; the communication-simulation library Sionna and the FEC tool AFF3CT also make extensive use of discrete probability and finite-blocklength simulation.
8. Discrete Differential Geometry: Discrete exterior calculus and discrete curvature already form an engineering toolchain. [Discrete Differential Geometry](#) course materials, [libigl](#), [geometry-central](#), and the 2026 [Dxtr](#) library are all useful reference examples.
9. Discrete Dynamical Systems and Integer AI Computation: Integer inference has entered mainstream deployment. The [LiteRT/TensorFlow Lite 8-bit quantization spec](#), [ONNX Runtime quantization](#), and PyTorch quantization tools all lower neural-network inference to int8/int16/fixed-point arithmetic.
10. Discrete Complex Analysis and Dual Numbers: Complex numbers and dual numbers appear as program objects in automatic differentiation systems. Automatic differentiation in JAX, PyTorch, and TensorFlow can be viewed as the chain rule implemented over finite computation graphs; finite-field extensions are used heavily in arkworks, SageMath, FLINT, and zk proof systems.
11. Discrete Differential Equations: Equation solvers actually executed in engineering are mostly discrete recurrences. [SciPy integrate](#), [DifferentialEquations.jl](#), and [do-mpc](#) all discretize continuous models into finite-step iterations or finite-horizon optimization.
12. Discrete Optimization and Control: [Google OR-Tools](#) provides CP-SAT, integer programming, graph algorithms, and scheduling tools; [JuMP](#) supports integer programming, conic programming, and nonlinear optimization; [CVXPY](#) also includes interfaces for mixed-integer problems.
13. Discrete Information Theory and Coding: [AFF3CT](#) supports simulations of Turbo, LDPC, Polar, and other forward-error-correcting codes; [NVIDIA Sionna](#) targets 5G/6G communication links, system-level simulation, and machine-learning communications; LDPC and Polar codes in 5G NR are already standardized engineering examples.

14. From Mathematical Definitions to Instruction Sets: The [RISC-V ISA manual](#) shows how an open instruction set precisely defines integer instructions, exceptions, and state transitions; [Sail](#) can be used to write ISA semantics; proof chains for CompCert, CakeML, and seL4 also rely on precise machine models.
15. Discrete Topology and Combinatorial Homology: [GUDHI](#), [Ripser](#), [Dionysus](#), and [giotto-tda](#) have already made simplicial complexes, persistent homology, and topological machine learning into usable software.
16. Algebraic Geometry over Finite Fields: [Macaulay2](#) and [Singular](#) are classic systems for algebraic geometry, commutative algebra, and polynomial computation; SageMath integrates finite-field curves, coding, elliptic curves, and polynomial ideals into a unified environment.
17. Constructive Mathematics and Type Theory: [Rocq Prover](#), [Lean 4 / mathlib](#), [Isabelle](#), and [Dafny](#) all use types, inductive definitions, specifications, and proof objects to express checkable mathematics and programs.
18. Discrete Automatic Differentiation: JAX, PyTorch, TensorFlow, Enzyme, MLIR autodiff, and related systems transform programs into finite computation graphs or derivative propagation over intermediate representations. Integer training still requires approximate gradients, but integer inference, fixed-point backpropagation, and quantization-aware training already have industrial tool support.
19. From Definitions to Formal Verification: [Project Everest](#), [HACL*](#), [Fiat-Crypto](#), [seL4](#), [CompCert](#), and [CakeML](#) are mature examples of moving “from specification to implementation.”
20. Discrete Stochastic Processes and Martingales: Binomial-tree option pricing, finite Markov chains, queueing networks, and discrete-time financial models all already have tool support. QuantLib, Sionna, NumPy/SciPy, and probabilistic-programming systems can serve as implementation references.
21. Discrete Metric Spaces: [NetworkX](#) provides graph-metric algorithms such as shortest paths, connectivity, centrality, and clustering; nearest-neighbor libraries such as Annoy, FAISS, and hnswlib engineer high-dimensional distance search, and although some internals use floating point, the index structures themselves are discrete graphs or trees.
22. Finite-Dimensional Operator Algebras: Sparse matrices, graph Laplacians, and finite-dimensional linear operators are implemented in SciPy, SuiteSparse, Eigen, PETSc, and GraphBLAS. [SuiteSparse:GraphBLAS](#) is especially close to the route of “algebraic structures + graph algorithms.”
23. Discrete Signal Processing: SciPy Signal, FFTW, KFR, LiquidDSP, GNU Radio, and related tools implement FIR/IIR filters, decimation, interpolation, FFT, and communication signal processing. Integer wavelets and lifting schemes already have engineering applications in standards such as JPEG 2000.
24. DCA-ISA and Microarchitecture Sketch: RISC-V custom instructions, MLIR/IREE compiler stacks, TVM, XLA, Triton, and various NPU/TPU compilers are all mapping “from tensors/polynomials/loops to low-level machine operations.” [IREE](#) explicitly targets deployment from data centers to mobile and embedded devices.
25. Operating Systems and Certified Kernels: seL4 is the clearest example. It has machine-checked proofs of kernel correctness and maintains proof stacks for Arm, RISC-V, Intel, and other architectures. TLA+ and TLC are also often used for finite-state modeling of distributed systems and OS protocols.
26. Finite-Field Quantum Models: Mainstream quantum software such as [Qiskit](#), [Cirq](#), [QuTiP 5](#), and [Stim](#) all turn quantum circuits, stabilizer circuits, or open-system simulation into finite data structures and finite-step computation.
27. Discrete Spacetime and Causal Cellular Automata: Lattice field theory, finite-difference time-domain methods, lattice Boltzmann methods, and cellular automata are all existing examples. Golly can directly explore rules such as the Game of Life, von Neumann automata, WireWorld, and Langton loops.
28. Computational Complexity: Complexity theory itself takes finite input strings and resource bounds as objects. SAT Competition, SMT-COMP, DIMACS formats, QBF solvers, and model-checking benchmarks are all ways of turning theoretical problems into runnable instances.
29. Cellular Automata and Computational Universality: [Golly](#) is an open cellular-automata exploration tool; Rule 110 universality, Game of Life Turing machines, and WireWorld logic gates are classic examples of “local discrete rules implementing computation.”
30. Formal-Verification Loop: TLA+ toolchains, Dafny, SPARK Ada, Frama-C, Why3, and Kani Rust Verifier are engineering tools for specification-verification-implementation loops. [Kani](#) performs bit-precise model checking for Rust and is

especially close to finite-state verification.

31. Discrete Information Geometry: Finite distributions, total-variation distance, fixed-point approximations of KL, and discrete Fisher information have all been implemented in machine learning and statistical inference. scikit-learn, PyTorch, JAX, and probabilistic-programming systems can serve as experimental platforms.
32. Symbolic Dynamics and Discrete Chaos: Subshifts, finite automata, modular-integer cat maps, and pseudorandom scrambling commonly appear in image encryption, symbolic-sequence analysis, and dynamical-systems teaching. NetworkX can be used to turn subshifts of finite type into directed-graph path problems.
33. Discrete Optimal Control: Finite-horizon dynamic programming, MPC, and mixed-integer MPC have mature tools. do-mpc, CasADi, JuMP, OR-Tools, and cvxpy can express discrete or discretized control problems at different levels.
34. Algebraic Coding and Cryptography: The NIST 2024 standards [FIPS 203 ML-KEM](#), [FIPS 204 ML-DSA](#), and [FIPS 205 SLH-DSA](#) are among the most important recent engineering examples in post-quantum cryptography; zk systems such as arkworks, Winterfell, and Plonky2 also rely on finite fields, polynomials, and coding ideas.
35. The Expressive Scope of DCA: The computational-mathematics software ecosystem is itself evidence of scope. SageMath, GAP, FLINT, Singular, Macaulay2, NetworkX, OR-Tools, GUDHI, and related tools cover discrete algebra, number theory, optimization, graphs, and topology.
36. Physical Implementation Blueprint: RISC-V, CHERI, seL4, MLIR/IREE, TVM, open-source EDA tools, and formal hardware verification all bring high-level specifications down to machine structures. This is better written as an “implementation roadmap” than as a claim about new physics.
37. Discrete Differential Topology and Morse Theory: Forman’s discrete Morse theory has been developed for many years; GUDHI, Dionysus, giotto-tda, and related TDA tools use critical simplices, filtrations, and persistent homology for data analysis.
38. Discrete Spectral Theory: Graph Laplacians, spectral clustering, PageRank, random walks, and graph neural networks are all built on finite matrices. NetworkX, PyTorch Geometric, DGL, and SciPy sparse are common implementation platforms.
39. Discrete Neural Architecture Search: NAS-Bench series, NNI, Optuna, Ray Tune, and TVM auto-scheduler all turn architectures, operator choices, latency, and memory budgets into finite search or optimization problems.
40. DCA Bootstrap Interpreter: Self-interpreters, bytecode VMs, WebAssembly, CakeML’s bootstrapped compiler, and Racket/Scheme teaching interpreters are all useful references. CakeML is especially useful as a reference for a “verified language implementation.”
41. Discrete Physical Mapping: Lattice gauge theory, lattice Boltzmann methods, FDTD, electromagnetic and elastic simulation, and cellular-automaton physical models all have software and literature foundations. This document should treat them as numerical-model and computation-model examples.
42. Automated Theorem Proving: SAT/SMT, E-prover, Vampire, Lean automation, Rocq tactics, Isabelle Sledgehammer, and cvc5 proof output are all usable routes. In recent years, LLMs have also begun assisting Lean/Isabelle proof search, but proofs still need to be checked by the proof assistant.
43. Fully Discrete Agent Concept: Auditable agents can draw on planners, model checkers, rule engines, integer inference models, and safety-constraint systems. A more realistic target is “finite-state planning + quantized model + formal safety shell,” rather than directly claiming AGI.

Table of Contents

Part 1: Arithmetic Foundations Part 2: Algebraic Structures Part 3: Discrete Analysis Part 4: Discrete Geometry Part 5: Logic and Reasoning Part 6: Discrete Fourier Analysis and Number-Theoretic Transforms Part 7: Discrete Probability Part 8: Discrete Differential Geometry Part 9: Discrete Dynamical Systems and Integer AI Computation Part 10: Discrete Complex Analysis and Dual Numbers Part 11: Discrete Differential Equations Part 12: Discrete Optimization and Control Part 13: Discrete Information Theory and Coding Part 14: From Mathematical Definitions to Instruction Sets Part 15: Discrete Topology and Combinatorial Homology Part 16: Algebraic Geometry over Finite Fields Part 17: Constructive Mathematics and Type Theory Part 18: Discrete Automatic Differentiation Part 19: From Definitions to

Formal Verification Part 20: Discrete Stochastic Processes and Martingales Part 21: Discrete Metric Spaces Part 22: Finite-Dimensional Operator Algebras Part 23: Discrete Signal Processing Part 24: DCA-ISA and Microarchitecture Sketch Part 25: Operating Systems and Certified Kernels Part 26: Finite-Field Quantum Models Part 27: Discrete Spacetime and Causal Cellular Automata Part 28: Computational Complexity Part 29: Cellular Automata and Computational Universality Part 30: Formal-Verification Loop Part 31: Discrete Information Geometry Part 32: Symbolic Dynamics and Discrete Chaos Part 33: Discrete Optimal Control Part 34: Algebraic Coding and Cryptography Part 35: The Expressive Scope of DCA Part 36: Physical Implementation Blueprint Part 37: Discrete Differential Topology and Morse Theory Part 38: Discrete Spectral Theory Part 39: Discrete Neural Architecture Search Part 40: DCA Bootstrap Interpreter Part 41: Discrete Physical Mapping Part 42: Automated Theorem Proving Part 43: Fully Discrete Agent Concept

Part 1: Arithmetic Foundations

Definition

Let $B=\{0,1\}$. A w -bit unsigned integer is a bit string $a=(a_{w-1},\dots,a_0) \in B^w$, whose value is $\text{val}(a)=\sum_{i=0}^{w-1} a_i 2^i$. Machine addition is defined as addition modulo 2^w :

$$a +_w b = (\text{val}(a) + \text{val}(b)) \bmod 2^w.$$

In a bitwise implementation, the i -th sum bit and carry satisfy:

$$s_i = a_i \text{ xor } b_i \text{ xor } c_i, \quad c_{i+1} = \text{majority}(a_i, b_i, c_i).$$

Subtraction can be defined as two's-complement addition: $a -_w b = a +_w ((\sim b) +_w 1)$. Multiplication is defined as a finite sum of shift-add operations:

$$a *_w b = \sum_{i: b_i=1} (a \ll i) \bmod 2^w.$$

Division should not be written as real-number division. It should return a quotient-remainder pair (q,r) satisfying $a=bq+r$ and $0 \leq r < b$. At machine word length, if intermediate overflow must be prevented, a wider word length or arbitrary-precision integers should be used.

Argument

Correctness of the adder can be proved by induction on bits. The induction invariant is: after the low k bits have been processed, the result already written equals $(a \bmod 2^k + b \bmod 2^k) \bmod 2^k$, and c_k is exactly the carry into bit k . The case $k=0$ holds. From k to $k+1$, xor gives the parity of the current bit, while majority gives the carry when at least two inputs are 1, so the invariant is preserved. After processing w bits, we obtain the sum modulo 2^w .

Correctness of multiplication follows from binary expansion: $b = \sum b_i 2^i$, so $ab = \sum b_i (a 2^i)$. Each nonzero bit triggers exactly one shift-add operation, so the process is finite. Division can be proved correct using the long-division invariant: when testing quotient bits from high to low, we always maintain original $a = bq + r$ and $r \geq 0$; at the end, if no bit can continue subtracting b , then $r < b$.

Implementation Notes

A program should distinguish three semantics explicitly: modular wraparound, saturation, and arbitrary precision. Many mathematical properties depend on this choice. For example, w -bit wraparound addition forms an abelian group; saturating addition usually does not satisfy associativity; arbitrary-precision integers satisfy the usual properties of the integer ring. In an open-source implementation, it is advisable to separate $\text{Word}[w]$, Nat , Int , and $\text{Mod}[p]$ into different types, so the same $+$ does not hide multiple sets of rules.

Part 2: Algebraic Structures

Definition

A finite set is a duplicate-free finite list of enumerable objects, usually written $A = \{a_0, \dots, a_{n-1}\}$. A function $f: A \rightarrow B$ can be represented at the implementation layer by a table, a program, or a combination of both. If it is represented by a program, finite-step termination must be guaranteed for every input in A .

A group is a structure with a binary operation $\ast: G \times G \rightarrow G$ and an identity element e , satisfying closure, associativity, identity, and inverses. A finite ring is a finite set with addition and multiplication, satisfying an additive group structure, associativity of multiplication, and distributivity. A finite field is a finite ring whose nonzero elements also form a group under multiplication.

A typical example is $\mathbb{Z}/n\mathbb{Z}$. When n is prime, $\mathbb{Z}/p\mathbb{Z}$ is a field; when n is composite, zero divisors exist, so it is usually not a field. w -bit machine integers under wraparound addition and multiplication are isomorphic to $\mathbb{Z}/2^w\mathbb{Z}$; this is a finite commutative ring, but when $w \geq 2$ it is not a field.

Argument

Properties of finite algebraic structures can be mechanically verified. Given a multiplication table for G , closure is determined by whether every table entry remains in G ; associativity can be checked by enumerating all triples (a, b, c) and testing $(a \ast b) \ast c = a \ast (b \ast c)$; identity and inverses can be found by finite search. In a finite field, every nonzero element has an inverse. This can be proved by the extended Euclidean algorithm: if $\gcd(a, p) = 1$, then there exist integers u, v such that $ua + vp = 1$, so u is the inverse of a modulo p .

Matrices, vectors, and linear maps here are all finite-dimensional. Associativity of matrix multiplication follows from rebracketing finite sums:

$$((AB)C)_{ij} = \sum_k (\sum_l A_{il} B_{lk}) C_{kj} = \sum_l A_{il} (\sum_k B_{lk} C_{kj}) = (A(BC))_{ij}.$$

If the underlying ring is a modular ring, every equality is interpreted under the same modulus.

Implementation Notes

Algebraic structures should be expressed as “carrier set + operations + property checkers” whenever possible. For small structures, direct enumeration can verify properties; for large finite fields, algorithmic proof can replace full enumeration. Matrix libraries need to explicitly record dimensions, the underlying ring, the modulus, and the overflow policy; otherwise the same code may belong to different mathematical structures.

Part 3: Discrete Analysis

Definition

A discrete function is a map on a finite integer interval or finite set. For $f: \{a, \dots, b+1\} \rightarrow \mathbb{R}$, the forward difference is defined as:

$$\Delta f(x) = f(x+1) - f(x).$$

Finite summation is defined as:

$$\sum_{x=a}^b f(x) = f(a) + f(a+1) + \dots + f(b).$$

Here “integration” is only finite summation and involves no limits. If the step size is h , one may define $\Delta_h f(x) = f(x+h) - f(x)$. If a continuous derivative is to be approximated, the scaling factor $1/h$ and an error bound should be stored separately.

Argument

The basic identity of discrete calculus is:

$$\sum_{x=a}^b \Delta f(x) = f(b+1) - f(a).$$

The proof is telescoping cancellation:

$$(f(a+1) - f(a)) + (f(a+2) - f(a+1)) + \dots + (f(b+1) - f(b)) = f(b+1) - f(a).$$

The entire proof consists only of finite-term rearrangement, addition, and subtraction, so it applies to integers, rational numbers, finite fields, or modular rings. If working in a modular ring, the equality holds modulo the modulus; if working at machine word length, the equality holds under wraparound semantics.

Implementation Notes

When discrete analysis is used in numerical programs, the “definition layer” and the “approximation layer” should be separated. Differences, convolutions, and finite sums at the definition layer are exact discrete objects; only the approximation layer discusses their relationship to continuous derivatives, integrals, or differential equations. This avoids mistaking floating-point error for mathematical error, and it also makes it easier to migrate the same algorithm to integer hardware.

Part 4: Discrete Geometry

Definition

The basic objects of discrete geometry are finite grids, finite graphs, or finite simplicial complexes. A grid point can be written as an integer vector $p = (p_1, \dots, p_d) \in \mathbb{Z}^d$. Common distances include:

$d_1(p, q) = \sum_i |p_i - q_i|$, the Manhattan distance; $d_H(x, y) = \text{popcount}(x \oplus y)$, the Hamming distance; graph distance $d_G(u, v)$, the number of edges in a shortest path.

A discrete line is a finite sequence of points satisfying an adjacency relation. A discrete circle can be defined as $\{p: d(p,c)=r\}$. Discrete area or volume is a count of cells, grid points, or simplices in a region, depending on the modeling convention.

Argument

Manhattan distance satisfies the metric axioms. Nonnegativity and symmetry are immediate. The triangle inequality follows from the integer absolute-value inequality:

$$|p_i - r_i| \leq |p_i - q_i| + |q_i - r_i|.$$

Summing over all dimensions gives $d_1(p,r) \leq d_1(p,q) + d_1(q,r)$. The triangle inequality for graph distance also follows directly from path concatenation: a shortest path from u to v and a shortest path from v to w concatenate into a path from u to w , so the shortest distance cannot exceed the length of that concatenated path.

Implementation Notes

In DCA, Euclidean irrational lengths should not be the default geometric quantity. If Euclidean geometry is needed, one can use integer squared distance $\|p-q\|^2$, or use rational approximation with an explicit error. For path planning, image processing, grid computation, and chip routing, Manhattan distance, Chebyshev distance, and graph distance are often closer to the implementation.

Part 5: Logic and Reasoning

Definition

Truth values of propositions are taken from $B=\{0,1\}$. The basic logical operations are defined as:

$$\text{NOT } p = 1 \text{ xor } p, \text{ AND}(p,q) = p \& q, \text{ OR}(p,q) = p | q, p \rightarrow q = \text{OR}(\text{NOT } p, q).$$

A propositional formula with n Boolean variables can be viewed as a function $B^n \rightarrow B$. If the formula evaluates to 1 for every input, it is a tautology; if it evaluates to 1 for at least one input, it is satisfiable.

Argument

The decision procedure for finite propositional logic is direct: enumerate the 2^n assignments and evaluate the formula. Correctness follows from the definition itself, because every possible world is enumerated exactly once. If the formula is represented in conjunctive normal form, SAT solvers can use DPLL or CDCL-style algorithms to avoid full enumeration, but these algorithms still maintain a finite search tree. Every propagation, branching, and backtracking operation acts on a finite variable set.

Inference rules can also be verified by tautologies. For example, modus ponens corresponds to the formula $((p \rightarrow q) \text{ AND } p) \rightarrow q$. Enumerating the four cases $p,q \in B$, the formula is always 1, so the rule is valid.

Implementation Notes

Turning proofs into Boolean formulas is an important DCA workflow. Small propositions can be verified directly by truth tables; large problems can be sent to SAT/SMT solvers; program properties can first be turned into finite-state or bounded model-checking problems. It should be noted that a “decidable fragment” does not mean all mathematical propositions are automatically decidable; the boundary must be stated clearly.

Part 6: Discrete Fourier Analysis and Number-Theoretic Transforms

Definition

For a finite sequence $a=(a_0,\dots,a_{n-1})$ of length n , if an n -th primitive root of unity ω exists in the finite field F_p , the number-theoretic transform NTT is defined as:

$$A_k = \sum_{j=0}^{n-1} a_j \omega^{jk} \bmod p.$$

The inverse transform is:

$$a_j = n^{-1} \sum_{k=0}^{n-1} A_k \omega^{-jk} \bmod p.$$

The usual existence condition is that $n \mid (p-1)$ and n is invertible in F_p . A commonly used prime such as 998244353 is popular because it supports long power-of-two transform lengths.

Argument

Correctness of the inverse transform rests on the orthogonality relation in a finite field:

$$\sum_{k=0}^{n-1} \omega^{k(j-l)} = n \text{ when } j=l, \text{ and } 0 \text{ otherwise.}$$

If $j \neq l$, let $r = \omega^{j-l}$. Then $r^n = 1$ and $r \neq 1$. The finite geometric sum satisfies $(r-1) \sum_{k=0}^{n-1} r^k = r^n - 1 = 0$; since $r-1$ is invertible, the sum is 0. Substituting this into the inverse transform formula leaves only the a_j term, so the inverse transform is correct.

The convolution theorem also follows by changing the order of finite summation: circular convolution of sequences becomes pointwise multiplication in the NTT domain. There is no floating-point error here, only choices of modulus, length conditions, and overflow policies.

Implementation Notes

NTT is a common tool in integer convolution, homomorphic encryption, polynomial multiplication, and post-quantum cryptography. An open-source implementation should state the prime, primitive root, maximum length, bit-reversal strategy, and normalization convention for negative values. If CRT is used to combine multiple moduli, the final integer range and reconstruction conditions should also be stated.

Part 7: Discrete Probability

Definition

A finite probability space consists of a sample space $\Omega=\{\omega_1,\dots,\omega_N\}$ and a probability mass function $P:\Omega\rightarrow\mathbb{Q}_{\geq 0}$, satisfying $\sum_{\omega} P(\omega)=1$. If counting probability is used, then $P(E)=|E|/|\Omega|$.

A random variable is a function $X:\Omega\rightarrow\mathbb{R}$. Expectation is defined as a finite sum:

$$E[X]=\sum_{\omega\in\Omega} X(\omega)P(\omega).$$

Conditional probability, when $P(B)>0$, is defined as:

$$P(A|B)=P(A\cap B)/P(B).$$

Argument

Bayes' formula follows from commutativity of multiplication:

$$P(A|B)P(B)=P(A\cap B)=P(B|A)P(A).$$

As long as the denominator is nonzero, we get $P(A|B)=P(B|A)P(A)/P(B)$. If all probabilities are stored as integer weights, for example as rational numbers with a common denominator, the entire derivation only involves integer addition, multiplication, and reduction.

A discrete Markov chain is given by a finite state set and a transition matrix. The t -step distribution is the initial row vector multiplied on the right by P^t . If states and probabilities are represented as rational numbers, matrix powers remain finite algebraic operations.

Implementation Notes

Programs should use fractions or integer weights whenever possible, rather than default floating-point probabilities. For large-scale models, fixed-point approximations can be used, but the scaling factor must be preserved. Limit theorems in probability can serve as external analysis tools; the implementation layer usually uses finite-sample deviation bounds, confidence intervals, or combinatorial inequalities.

Part 8: Discrete Differential Geometry

Definition

Discrete differential geometry represents curves, surfaces, and manifolds as finite grids, triangular meshes, or simplicial complexes. A discrete curve is a point sequence $(p_0,\dots,p_m)$. Curvature can be represented by turning angles, direction changes, or local combinatorial quantities. On a triangular mesh, a common vertex curvature is angular defect:

$$K(v)=2\pi-\sum_{f\ni v} \theta_f(v).$$

If full integerization is desired, one can use combinatorial curvature on a regular triangular mesh, such as $K_c(v)=6-\deg(v)$, where $\deg(v)$ is the vertex degree.

Argument

The reasonableness of combinatorial curvature comes from the local flat model. In a regular triangular mesh, an interior flat vertex usually has 6 triangles around it, so curvature is 0 when $\deg(v)=6$; fewer than 6 indicates positive curvature, and more than 6 indicates negative curvature. For a closed triangulation, the Euler characteristic

$$\chi = V - E + F$$

is an integer topological invariant. Discrete Gauss-Bonnet-type results connect local curvature to global topology. If angular defect is used, total curvature is $2\pi\chi$; if normalized combinatorial curvature is used, an equivalent integer identity can also be obtained.

Implementation Notes

Geometry-processing programs should record vertex tables, edge tables, face tables, and adjacency relations. Continuous concepts such as curvature, normals, and geodesic distance can be approximated with discrete operators, but mesh quality and error sources must be stated. Discrete exterior calculus gives a more systematic approach: differential forms are placed on simplices, and exterior differentiation becomes the transpose of the boundary matrix.

Part 9: Discrete Dynamical Systems and Integer AI Computation

Definition

A discrete dynamical system is a state recurrence:

$$x_{t+1} = F(x_t),$$

where the state space S is a finite set or a finitely encoded set, and $F: S \rightarrow S$ is computable. If S is finite, every orbit must eventually enter a cycle.

Integer AI computation writes neural-network layers as integer or fixed-point functions. For example, a linear layer can be written as $y = Wx + b$, where W, x, b are all integers; an activation function can be $\text{ReLU}(x) = \max(0, x)$; normalization and softmax can be replaced by lookup tables, piecewise-linear functions, fixed-point exponentials, or sorting approximations.

Argument

The cyclicity of finite dynamical systems follows from the pigeonhole principle. If the first $|S|+1$ states in an orbit $x_0, x_1, \dots$ were all distinct, they would exceed the size of the state space, a contradiction. Therefore there exist $i < j$ such that $x_i = x_j$. Since the recurrence function is deterministic, the sequence thereafter repeats with period $j-i$.

Correctness of an integer neural network does not mean it is identical to a real-valued network. It means that under a given quantization rule, it exactly implements a discrete function. If the quantization map is $q(x) = \text{round}(x/s) + z$, then one must prove that the integer formula for each layer agrees with the quantized specification and provide an error bound.

Implementation Notes

In AI scenarios, training, inference, and verification should be distinguished. Inference can be fully integerized; training often still relies on approximate gradients, but fixed-point gradients or quantization-aware training can reduce the gap. For open-source releases, tensor ranges, scaling factors, overflow handling, and rounding rules should be written into the model metadata.

Part 10: Discrete Complex Analysis and Dual Numbers

Definition

If complex numbers are to be simulated in finite structures, finite-field extensions can be used. For a prime p , if -1 is not a square in F_p , one can construct $F_p[i]=F_p[x]/(x^2+1)$, with elements written as $a+bi$. If $p \equiv 1 \pmod{4}$, then i already exists in F_p .

Dual numbers are defined as $a+b\epsilon$, satisfying $\epsilon^2=0$. In a discrete implementation, a, b can be integers, modular integers, or rational numbers. If a function is built from addition and multiplication, then:

$$f(a+b\epsilon)=f(a)+b f'(a) \epsilon$$

corresponds to forward-mode automatic differentiation.

Argument

The differentiation rule for dual numbers can be proved by algebraic expansion. For multiplication:

$$(a+b\epsilon)(c+d\epsilon)=ac+(ad+bc)\epsilon+bd\epsilon^2=ac+(ad+bc)\epsilon.$$

This exactly corresponds to the product rule $(fg)'=f'g+fg'$. For polynomials and programs built from basic arithmetic, structural induction proves that the ϵ coefficient propagated by dual numbers equals the derivative or linearized difference result.

Closure of finite-field complex multiplication follows from the quotient-ring definition: every polynomial can be reduced modulo x^2+1 to a linear form $a+bi$. Thus addition and multiplication both land back in the same finite set.

Implementation Notes

This document does not directly transfer continuous results from complex analysis, such as holomorphicity or residue theorems, to finite fields. A safer approach is to understand “discrete complex analysis” as finite-field extensions, polynomial maps, and difference relations on grids. Dual numbers are more directly useful for automatic differentiation and program transformation.

Part 11: Discrete Differential Equations

Definition

In this document, a discrete differential equation means a difference equation or recurrence system. A linear constant-coefficient difference equation can be written as:

$$x_{t+k} = a_{k-1}x_{t+k-1} + \dots + a_0x_t + u_t.$$

If we define the state as $s_t = (x_t, x_{t+1}, \dots, x_{t+k-1})$, it becomes a first-order system:

$$s_{t+1} = A s_t + b u_t.$$

If variables take values in a finite ring or finite field, the system is a finite-state machine. If variables are bounded integers, it can still be computed exactly within a finite window.

Argument

Converting a high-order recurrence into a first-order recurrence is justified by state expansion. s_t stores all the history values needed to compute the next term; the first few rows of matrix A perform shifting, and the last row substitutes the recurrence formula. By induction, for every t , the first component produced by the matrix system is the same as x_t produced by the original difference equation.

If the state space is finite, the solution orbit eventually cycles. If the matrix is invertible over a finite field, the state transition is a permutation, and every orbit lies in a cycle from the beginning.

Implementation Notes

Engineering documents should first make clear the sampling period, state vector, input bounds, and numerical range. Many solvers for continuous differential equations ultimately land on difference equations. The DCA perspective does not deny continuous models; it requires the actually executed discrete recurrence to be written as an independent and verifiable object.

Part 12: Discrete Optimization and Control

Definition

A discrete optimization problem can be written as:

minimize $f(x)$, where $x \in X$ and X is a finite set.

If constraints exist, they are written as $g_i(x) \leq 0$ or Boolean predicates $C_i(x) = 1$. A discrete control problem usually contains a finite state set S , action set A , transition function $T: S \times A \rightarrow S$, and cost function $c: S \times A \rightarrow \mathbb{Z}$.

The value function for finite-horizon dynamic programming is defined as:

$$V_t(s) = \min_{a \in A(s)} [c(s, a) + V_{t+1}(T(s, a))],$$

with terminal condition $V_T(s) = h(s)$.

Argument

Correctness of dynamic programming follows from optimal substructure. Any policy starting at time t first chooses some action a , and the remaining part is a policy from $T(s, a)$ to the terminal time. If the remaining policy were not optimal for that subproblem, it could be replaced by a better policy, improving the whole policy, a contradiction.

Therefore the optimal value satisfies the Bellman recurrence.

Because S , A , and time T are all finite, value iteration requires only finitely many computation steps. If costs are integers, the process contains only integer addition, comparison, and minimization.

Implementation Notes

Discrete optimization does not necessarily require gradients. Enumeration, branch and bound, dynamic programming, integer programming, and SAT/SMT encodings are all natural tools. In control problems, if continuous states are used, the discretization grid and error should be stated. If the state is already a program state, inventory level, task queue, or graph node, the DCA form can be used directly.

Part 13: Discrete Information Theory and Coding

Definition

An information source is a symbol-sequence generation mechanism over a finite alphabet Σ . If symbol x has probability p_x , Shannon entropy is:

$$H(X) = -\sum_x p_x \log(1/p_x).$$

At the implementation layer, p_x can be stored as integer counts, and logarithm tables can be stored as fixed-point numbers. A code is a map $C: \Sigma \rightarrow \{0,1\}^*$. If no codeword is a prefix of another, the code is called a prefix code.

An error-correcting code is a set $C \subset \Sigma^n$. The minimum distance $d_{\min}$ is the minimum Hamming distance between distinct codewords. If $d_{\min} \geq 2t+1$, then up to t errors can be corrected.

Argument

The error-correction capability proof is direct. If a received word y is within distance at most t of the true codeword c , and also within distance at most t of another codeword c' , then by the triangle inequality:

$$d(c, c') \leq d(c, y) + d(y, c') \leq 2t.$$

This contradicts $d_{\min} \geq 2t+1$. Therefore balls of radius t around codewords are disjoint, and nearest-neighbor decoding is unique.

A prefix code can be decoded immediately because once the input bitstream matches some codeword, it cannot merely be the prefix of a longer codeword. Decoding is a finite tree traversal.

Implementation Notes

Coding theory is highly suitable for DCA: alphabets, codewords, parity-check matrices, syndromes, and decoding tables are all finite objects. Places that require real numbers are usually performance analyses, such as capacity and entropy. The implementation layer can use integer counts, fixed-point logarithms, and finite-blocklength bounds.

Part 14: From Mathematical Definitions to Instruction Sets

Definition

An instruction set is the machine interface of mathematical operations. A simplified DCA-ISA should contain at least: bit logic AND/OR/XOR/NOT, integer arithmetic ADD/SUB/MUL/DIV, shifts SHL/SHR/SAR, comparison CMP, memory access LOAD/STORE, and control flow JMP/BRANCH.

To support the algorithms above, one can extend it with:

MAJ for full-adder carry acceleration; MADD for matrix multiplication and convolution; BITREV for NTT; POPCNT for Hamming distance; MIN/MAX for optimization and ReLU.

Argument

The key step from mathematical definitions to instruction sets is semantic preservation. For example, the specification of instruction $\text{ADD}_w(x,y)$ is to return $(x+y) \bmod 2^w$. If the hardware circuit is composed of chained full adders, then the bit-induction proof in Part 1 also proves that this instruction implements the specification.

For composite programs, proof can use compositionality: if every instruction satisfies its local specification, and if the state transitions of registers, memory, and program counter are clearly defined, then the execution trace of a program satisfies the global specification assembled from those local specifications.

Implementation Notes

Open source does not require designing a real chip, but it should write the “operational semantics” clearly. Documentation for open instruction sets such as RISC-V is a useful reference: every instruction has an encoding, inputs, outputs, exceptions, and boundary behavior. DCA documents should do the same, avoiding the mixing of “mathematical addition” and “machine addition.”

Part 15: Discrete Topology and Combinatorial Homology

Definition

A simplicial complex consists of vertices, edges, triangles, and higher-dimensional simplices, and satisfies: if a simplex belongs to the complex, all of its faces also belong to the complex. A k -chain is an integer- or mod- p -coefficient formal sum of k -dimensional simplices. The boundary operator $\partial_k: C_k \rightarrow C_{k-1}$ maps a simplex to its signed boundary.

The homology group is defined as:

$$H_k = \ker(\partial_k) / \text{im}(\partial_{k+1}).$$

Intuitively, $\ker(\partial_k)$ is the set of k -dimensional cycles without boundary, and $\text{im}(\partial_{k+1})$ is the set of boundaries of one-dimensional-higher objects.

Argument

The core identity of homology is $\partial_{k+1} \partial_k = 0$, meaning “the boundary of a boundary is empty.” For a triangle $[a,b,c]$, for example:

$$\partial [a,b,c] = [b,c] - [a,c] + [a,b].$$

Taking the boundary again, all vertex terms cancel pairwise. Thus every higher-dimensional boundary is a cycle, $\text{im}(\partial_{k+1}) \subset \ker(\partial_k)$, and the quotient structure is well-defined.

Betti numbers can be computed from ranks of boundary matrices:

$$\beta_k = \dim \ker(\partial_k) - \dim \text{im}(\partial_{k+1}).$$

Over a finite field, this is finite-matrix Gaussian elimination.

Implementation Notes

Topological data analysis, mesh connectivity, and hole detection can all be written as finite linear-algebra problems. Implementation requires a consistent vertex ordering and simplex orientation; otherwise the signs in the boundary matrix will be wrong. If only mod-2 homology is needed, orientation signs can be omitted, making computation simpler.

Part 16: Algebraic Geometry over Finite Fields

Definition

In DCA, algebraic geometry can first be restricted to finite fields F_q . Given a set of polynomials $f_1, \dots, f_m \in F_q[x_1, \dots, x_n]$, its finite-field solution set is:

$$V(F_q) = \{x \in F_q^n : f_i(x) = 0 \text{ for all } i\}.$$

This is a finite set. Polynomial evaluation, addition, multiplication, and reduction are all performed in the finite field. Projective spaces, curves, and codes can also be represented using finite-field coordinates.

Argument

Finiteness of the solution set is immediate: F_q^n has only q^n points. To determine whether a point lies on the variety, evaluate every polynomial at the point. Direct enumeration requires at most $q^n m$ evaluations; for larger scales, Groebner bases, elimination, linear algebra, or specialized finite-field algorithms can reduce computation.

The discreteness of algebraic-geometry codes also comes from finite fields. Select rational points on a finite-field curve, and form a codeword from function values at those points. All values lie in the finite field, so encoding and parity-check matrices can be stored exactly.

Implementation Notes

Finite-field algebraic geometry does not require floating point, but it does require a very careful field implementation. One should record the field order, irreducible polynomial, element representation, multiplication table or multiplication algorithm. For cryptographic and coding applications, mathematical correctness and security

assumptions must be separated; the latter cannot be proved from DCA axioms alone.

Part 17: Constructive Mathematics and Type Theory

Definition

Constructive mathematics emphasizes that proving the existence of an object usually requires giving a method to construct it. Type theory treats propositions as types and proofs as terms of those types. Inductive types are generated by finite constructors, such as natural numbers:

```
Inductive nat :=  
| 0 : nat  
| S : nat -> nat.
```

Finite lists, finite trees, syntax trees, and proof trees can all be represented by inductive types. Recursive functions must structurally decrease or provide a termination argument.

Argument

Structural induction is the basic proof principle for inductive types. To prove that a property $P(n)$ holds for all natural numbers, it suffices to prove $P(0)$ and prove $P(n) \rightarrow P(S\ n)$. This is because natural-number terms can only be constructed by applying S finitely many times to 0 .

Program termination can be proved similarly. If recursive calls are always made on a smaller structure, such as the tail of a list or a subtree, infinite descent is impossible, so the function terminates in finitely many steps.

Implementation Notes

Proof assistants are well-suited for expressing the core conventions of DCA: bit strings, word lengths, finite vectors, matrices, and boundary matrices can all be types. Compared with paper proofs, formalized versions force the author to write out every implicit premise, such as equal dimensions, nonzero moduli, or invertible denominators.

Part 18: Discrete Automatic Differentiation

Definition

Automatic differentiation treats a program as a computation graph built from primitive operations. Forward mode can use dual numbers $a+b\varepsilon$; reverse mode stores a Wengert tape, that is, every intermediate variable and its dependency relation.

For integer or fixed-point programs, gradients are not necessarily derivatives in the real-valued sense. This document may use three alternatives:

1. formal derivatives for polynomial integer programs;
2. forward differences or finite differences for difference programs;
3. straight-through estimators or discrete surrogate gradients for quantized neural networks, explicitly identifying them as training heuristics.

Argument

Correctness of reverse mode comes from applying the chain rule in reverse topological order. The computation graph is a finite directed acyclic graph. Each node v is computed from its parent nodes, and the objective is L .

Backpropagation maintains an adjoint value $\text{adj}[v] = \partial L / \partial v$. Starting from the output and processing in reverse order, if u is a parent of v , it accumulates:

$$\text{adj}[u] += \text{adj}[v] * \partial v / \partial u.$$

Because every path from u to the output is considered once during reverse processing, the final $\text{adj}[u]$ equals the sum of all path contributions.

Implementation Notes

Integer automatic differentiation requires explicit rounding and nondifferentiable-point conventions. ReLU at 0, rounding functions, and lookup-table functions are not ordinary smooth functions. Open-source implementations should separately describe training gradients and inference-time integer functions, avoiding the mistake of writing training approximations as inference semantics.

Part 19: From Definitions to Formal Verification

Definition

Formal verification writes objects, programs, and theorems into a proof system or checkable logic. Common DCA objects include:

bit, word w , finite vector $\text{Vec } A \ n$, matrix $\text{Mat } R \ m \ n$, graph $\text{Graph } n$, and program state State .

A specification is a predicate or relation, for example:

$$\text{add_spec}(a, b, r) := \text{val}(r) = (\text{val}(a) + \text{val}(b)) \bmod 2^w.$$

Implementation correctness is the theorem:

$$\forall a \ b, \text{add_impl}(a, b) \text{ satisfies } \text{add_spec}.$$

Argument

The verification chain usually has three layers. The first layer proves correctness of primitive operations such as full adders, multipliers, and comparators. The second layer proves correctness of algorithms such as matrix multiplication, NTT, and dynamic programming. The third layer proves correctness of system composition, such as compiler semantic preservation and instruction implementations satisfying ISA specifications.

Every layer relies on compositionality: if component specifications hold, and if the way components are connected satisfies interface conditions, then the overall specification holds. The value of formal tools is that they check whether interface conditions have been omitted.

Implementation Notes

This document does not require all content to be formalized immediately, but the writing style should make formalization easy. Every definition should preferably state its carrier, operations, assumptions, and output. Proofs should not merely say “obvious” ; they should identify whether the argument uses finite enumeration, induction, matrix algebra, or a loop invariant.

Part 20: Discrete Stochastic Processes and Martingales

Definition

A discrete-time stochastic process is a sequence of random variables $X_0, \dots, X_T$ defined on a finite probability space. A filtration F_t represents the information observable up to time t . If for all t :

$$E[X_{t+1}|F_t] = X_t,$$

then X_t is called a martingale. If the conditional expectation is greater than or equal to, or less than or equal to, the current value, it is a submartingale or supermartingale, respectively.

Argument

In the finite case, conditional expectation can be computed directly by summing over equivalence classes. Given an information block C of F_t , we have:

$$E[X_{t+1}|C] = \sum_{\{\omega \in C\}} X_{t+1}(\omega) P(\omega) / P(C).$$

If this value equals X_t on every information block, the martingale condition holds. The optional-stopping theorem in bounded time and finite state can be proved by repeatedly using conditional expectation: if $\tau \leq T$, freeze the process after the stopping time and obtain a new process $Y_t = X_{\min(t, \tau)}$. If X_t is a martingale, then Y_t is still a martingale, so $E[Y_T] = E[Y_0]$, i.e. $E[X_\tau] = E[X_0]$.

Implementation Notes

Financial binary trees, random walks, queueing systems, and online-algorithm analysis can all be expressed using finite stochastic processes. Probabilities should preferably be stored as integer weights or fractions. If a pseudorandom number generator is used, the generator should also be included in the state transition, rather than treating randomness as invisible external magic.

Part 21: Discrete Metric Spaces

Definition

A discrete metric space is a finite set X with a function $d: X \times X \rightarrow \mathbb{Q}_{\geq 0}$, satisfying:

1. $d(x, y) = 0$ if and only if $x = y$;
2. $d(x, y) = d(y, x)$;

$$3. d(x,z) \leq d(x,y) + d(y,z).$$

Common examples include the Hamming space $\{0,1\}^n$, weighted-graph shortest-path spaces, and L1 distance on integer grids. An isometric embedding is a map $\phi: X \rightarrow Y$ satisfying $d_X(x,y) = d_Y(\phi(x), \phi(y))$.

Argument

The triangle inequality for Hamming distance can be proved bit by bit. For any bit position, if $x_i \neq z_i$, then in the binary set it is impossible to have both $x_i = y_i$ and $y_i = z_i$; therefore this bit's contribution to $d(x,z)$ is no greater than its contribution to $d(x,y) + d(y,z)$. Summing over all bits gives the result.

The proof for graph shortest-path distance is the same as in Part 4, from path concatenation. Verification of an embedding is finite pair checking: enumerate all $x,y \in X$ and compare whether distances are preserved.

Implementation Notes

Discrete metric spaces are useful for nearest-neighbor search, clustering, error-correcting codes, graph embedding, and representation learning. Large-scale implementations need not store a complete distance matrix; they can store a generation rule or sparse adjacency list. If distances are only approximately preserved, a distortion bound α should be given, for example stating that $d_Y(\phi(x), \phi(y))$ lies between d_X and αd_X .

Part 22: Finite-Dimensional Operator Algebras

Definition

In DCA, operators can first be understood as finite matrices. Given a ring or field R , the set $\text{Mat}_n(R)$ of $n \times n$ matrices forms an algebra under matrix addition and multiplication. Concepts such as adjoint, norm, and spectrum need discrete substitutes if they involve real or complex numbers.

Usable integer norms include:

$$\|A\|_1 = \max_j \sum_i |A_{ij}|, \quad \|A\|_\infty = \max_i \sum_j |A_{ij}|.$$

If working over a finite field, “size” norms are no longer natural; the focus shifts more to rank, kernel, image, invariant factors, and characteristic polynomials.

Argument

Submultiplicativity of matrix multiplication under the 1-norm can be proved:

$$\|AB\|_1 = \max_j \sum_i |\sum_k A_{ik} B_{kj}|$$

$$\leq \max_j \sum_i \sum_k |A_{ik}| |B_{kj}|$$

$$= \max_j \sum_k |B_{kj}| \sum_i |A_{ik}|$$

$$\leq \|A\|_1 \|B\|_1.$$

This shows that finite-dimensional operators can be analyzed using integer inequalities. If rational or fixed-point numbers are used, the proof still remains valid.

Implementation Notes

One should not casually transfer infinite-dimensional C^* -algebra results to finite matrices. In engineering, it is more reliable to state matrix dimension, underlying field or ring, and norm. For quantum channels, Markov operators, and graph Laplacians, finite-matrix models are usually already sufficient to express the computable part.

Part 23: Discrete Signal Processing

Definition

A discrete signal is a finite sequence $x[0], \dots, x[N-1]$. A finite impulse-response filter is defined by convolution:

$$y[n] = \sum_{k=0}^{M-1} h[k]x[n-k].$$

Downsampling is $\text{downsample}_M(x) = x[0], x[M], x[2M], \dots$. Interpolation may first insert zeros and then filter. Wavelet transforms can be implemented by lifting schemes, where prediction, update, and rounding can all be integer operations.

Argument

Linearity of convolution follows from distributivity of finite sums:

$$h*(ax+bz) = a(h*x) + b(h*z).$$

The key in multirate systems is index transformation. Downsampling and convolution generally do not commute, so one must rederive from $y[m] = \sum_k h[k]x[mM-k]$. Reversibility of integer wavelet lifting comes from every step being an invertible integer transform. For example:

$$d = b - a, s = a + \text{floor}(d/2).$$

Given s, d , recover $a = s - \text{floor}(d/2)$ and $b = a + d$.

Implementation Notes

Sines, frequency responses, and spectrograms in signal processing are often used for analysis, but actual audio/video coding, image processing, and communication systems use many integer filters, fixed-point FFT/NTT operations, and lookup tables. Documents should distinguish floating-point visualizations used for analysis from integer pipelines used for implementation.

Part 24: DCA-ISA and Microarchitecture Sketch

Definition

DCA-ISA is a sketch of an instruction interface for the finite operations above; it is not a requirement to build new hardware. It can be viewed as a “minimal integer-computation target.” The state includes a register file, memory, program counter, and flags. Each instruction is a state-transition function:

step: $\text{State} \times \text{Instr} \rightarrow \text{State}$.

Microarchitecture is the circuit organization implementing this state transition, such as ALU, multiplier, divider, load-store unit, matrix-multiplication coprocessor, NTT coprocessor, and control unit.

Argument

ISA correctness and microarchitecture correctness are proved separately. The ISA layer defines abstract state transitions. The microarchitecture layer proves that when pipeline, forwarding, caches, and exception handling complete one instruction commit, the commit effect equals the ISA’s step. Such proofs usually use invariants: every uncommitted instruction in the pipeline has operands, destination registers, and ordering relations consistent with an abstract execution prefix.

For a coprocessor, such as an NTT unit, it is sufficient to prove that its input and output satisfy the transform specification from Part 6; upper layers need not know the internal butterfly schedule.

Implementation Notes

In practice, one can begin with a software simulator rather than hardware. First write an interpreter and tests, then map hot operations to SIMD, GPU, FPGA, or ASIC. As long as interfaces and specifications remain stable, the underlying implementation can be replaced gradually.

Part 25: Operating Systems and Certified Kernels

Definition

An operating-system kernel can be viewed as a finite-state machine. Its state includes task tables, page tables, scheduling queues, file-descriptor tables, permission bitmaps, and device states. A system call is a controlled state transition:

syscall: $\text{KernelState} \times \text{UserRequest} \rightarrow \text{KernelState} \times \text{Response}$.

A certified kernel requires specifications for key properties, such as memory isolation, permission checks, scheduling fairness, or interrupt handling that does not break kernel invariants.

Argument

A typical proof of memory isolation is an unreachability invariant. For any user process p , the set of addresses it may access is defined by page tables and permission bits. If every memory-access instruction first checks whether the address belongs to that set, and if system calls that modify page tables preserve the rule that authorized regions of different processes do not overlap illegally, then by induction, after any finite number of execution steps, process p still cannot read or write unauthorized pages.

Scheduler correctness can be proved using queue invariants: every ready task is either in the queue or currently running, and every context switch preserves this invariant.

Implementation Notes

The “mathematics” in OS work is not an abstract slogan. It consists of bitmaps, integer bounds, access-control matrices, and state machines. DCA-style writing is suitable for small kernels, embedded systems, and safety-critical components. The smaller the scope, the more feasible formalization becomes.

Part 26: Finite-Field Quantum Models

Definition

This part only discusses quantum-like linear models over finite fields; it does not claim to replace physical quantum mechanics. Let a state be a vector $\psi \in F_q^n$ over a finite field F_q . A gate is an invertible linear transformation $U \in GL_n(F_q)$. Measurement can be defined as a rule mapping states to a finite result set, for example by a function or by normalized discrete weights.

If one wants to simulate the structure of quantum circuits, tensor products, reversible gates, controlled gates, and oracle queries can be retained, but amplitudes are no longer complex probability amplitudes.

Argument

Invertible gates preserve information: if U is invertible, then from $\psi' = U\psi$ one can uniquely recover $\psi = U^{-1}\psi'$. A composition of invertible gates is still invertible because $(UV)^{-1} = V^{-1}U^{-1}$. Thus the evolution of a finite-field circuit is a permutation or invertible linear transformation on a finite set.

If one uses a discrete analogue of algorithms such as Deutsch-Jozsa, the interference or distinguishing property must be reproved in the chosen finite-field model; one cannot directly cite results from complex Hilbert spaces.

Implementation Notes

Finite-field quantum models are useful for studying reversible computation, linear-algebra toy models, and the structural skeletons of quantum algorithms. Genuine quantum probability, measurement postulates, and physical implementations require complex Hilbert spaces. This document retains only the finitely encodable part and puts the physical interpretation outside the boundary.

Part 27: Discrete Spacetime and Causal Cellular Automata

Definition

A discrete spacetime model takes space as integer grid points and time as integer steps. An event is written (t, x) . A local causal law can be defined as:

$\text{state}_{t+1}(x)$ depends only on $\text{state}_t(y)$ where $d(x, y) \leq r$.

Here r is the propagation radius. If $r=1$, information can propagate at most one cell per time step. A cellular automaton is a local rule:

$F: \text{local_neighborhood} \rightarrow \text{cell_state}.$

Argument

The causal-cone property can be proved by induction. At $t=0$, a point is affected only by itself. If after k steps, point x is affected only by initial points within distance at most kr , then at step $k+1$, the neighborhood points of x are each affected only by points within distance at most kr ; taking the union gives distance at most $(k+1)r$. Therefore finite-speed propagation holds.

Discrete wave, diffusion, or field equations written as local update rules can be proved to have finite domains of dependence in the same way.

Implementation Notes

Continuous Lorentz transformations from relativity cannot simply be integerized. The DCA version is better suited to expressing locality, finite propagation speed, conserved quantities, and simulability. If continuous physics is to be approximated, grid scale, stability conditions, and error analysis must be provided.

Part 28: Computational Complexity

Definition

A computational problem is a language $L \subseteq \{0,1\}^*$ or a function $f: \{0,1\}^* \rightarrow \{0,1\}^*$. The time complexity of an algorithm is a function $T(n)$ from input length n to a step-count bound; space complexity is similar.

The class P contains problems with polynomial-time algorithms. The class NP contains problems with polynomial-length certificates verifiable in polynomial time. NP -completeness is usually defined by polynomial-time reductions.

Argument

SAT belongs to NP because given a variable assignment, one can check whether every clause is satisfied in time polynomial in the formula length. If a problem A reduces to B in polynomial time, and B can be solved in polynomial time, then A can also be solved in polynomial time: first transform the input of A into an input of B , solve B , and interpret the answer back for A .

Asymptotic notation does not require processing an infinite object all at once. It expresses that there exists a computable bounding function such that every given input length has a finite resource bound.

Implementation Notes

Complexity analysis describes families of algorithms, not the measurement of a single run. DCA-style writing should provide both theoretical bounds and concrete implementation parameters. For fixed small-scale problems, exponential algorithms may be acceptable; for problems with growing input, resource bounds must be explicit.

Part 29: Cellular Automata and Computational Universality

Definition

A cellular automaton consists of a grid set, a finite state set, a neighborhood, and a local update rule. For a one-dimensional radius- r automaton:

$$x_{t+1}(i) = F(x_t(i-r), \dots, x_t(i+r)).$$

For the two-dimensional Game of Life, the state is alive/dead, and the next step is determined by the number of live cells among the 8 neighbors. Rule 110 is a one-dimensional binary cellular automaton whose local rule is given by an output table for the 8 possible three-cell neighborhoods.

Argument

The global update of a cellular automaton is a deterministic function on finite configurations or generatable configurations. If working on a finite toroidal grid, the number of states is finite, so every orbit must eventually cycle. If infinite but finitely supported configurations are allowed, then every finite number of steps affects only a finite causal cone.

Proofs of computational universality usually do not proceed by exhaustive enumeration. They construct a simulation: prove that an automaton can implement logic gates, storage, circuit synchronization, or tag systems, and then cite the equivalence of those models with Turing machines. The universality of Rule 110 is a result of this kind of constructive proof.

Implementation Notes

Cellular automata are very suitable for showing how simple local rules can generate complex global behavior. But open-source documentation should avoid directly interpreting complex behavior as intelligence or physical law. A reliable statement is: they provide an experimental platform for finite states, local updates, and computable simulation.

Part 30: Formal-Verification Loop

Definition

A verification loop means a cycle from specification, implementation, proof, and testing. A minimal loop includes:

1. mathematical specification Spec;
2. program implementation Impl;
3. theorem Impl refines Spec;
4. executable test cases;
5. versioned proof scripts or checking logs.

Compiler verification also requires semantic preservation: the semantics of the source program and the target program are consistent.

Argument

The core of the loop is a refinement relation. If there is a relation $R(A,C)$ between an abstract state A and a concrete state C , and if every concrete step corresponds to an abstract step or an allowed internal step, then the concrete implementation refines the abstract specification. Inductively, the initial state satisfies R , and every step preserves R , so every finite execution prefix satisfies the specification.

Matrix multiplication, sorting, parsers, and memory allocators can all be proved similarly: define an abstract specification, give a loop invariant, prove that each loop iteration preserves the invariant, and prove that termination satisfies the postcondition.

Implementation Notes

Testing should not be treated as proof, and proof should not be treated as performance testing. They complement one another: tests help discover specification misunderstandings and engineering errors, while proofs cover the theoretical state space. DCA documents should preferably give each core algorithm a small example, property tests, and a proof sketch.

Part 31: Discrete Information Geometry

Definition

A finite probability distribution can be represented as an integer weight vector $p=(p_1,\dots,p_n)$, with total weight $M=\sum p_i$. The normalized probability is p_i/M . The total-variation distance between two distributions can be written as:

$$TV(p,q)=1/2 \sum_i |p_i/M - q_i/M|.$$

If p,q have the same total weight, the integer form is:

$$TV_M(p,q)=1/2 \sum_i |p_i - q_i|.$$

Discrete Fisher information can replace derivatives with parameter differences, for example $\Delta_\theta \log p_\theta(x)$, and then take expectation by finite summation.

Argument

Total-variation distance satisfies the metric properties. Nonnegativity and symmetry are immediate; if the distance is 0, then every absolute-value term is 0, so $p=q$. The triangle inequality follows from the absolute-value triangle inequality:

$$|p_i - r_i| \leq |p_i - q_i| + |q_i - r_i|.$$

Summing and dividing by 2 gives the result. Since all quantities can be stored as integer weights, total variation is suitable as a basic distance between discrete probabilistic models.

Implementation Notes

Continuous information geometry relies on smooth manifolds and differential structure. In DCA, finite distribution families, integer weights, differences, and combinatorial divergences are more appropriate. If KL divergence is used, logarithms are needed; one can use rational bounds, fixed-point tables, or use it only in the analysis layer.

Part 32: Symbolic Dynamics and Discrete Chaos

Definition

Symbolic dynamics studies sequences over a finite alphabet and the shift map. Given alphabet Σ , the shift on a bi-infinite sequence space is:

$$(\sigma x)_t = x_{t+1}.$$

A subshift of finite type is defined by a set of forbidden words: all sequences that avoid the forbidden words form the system. If only finite windows are implemented, a state can be represented by a word of length k , and transitions are given by allowed concatenation relations.

A discrete chaotic map can be represented by a modular-integer map, such as the mod- N version of Arnold's cat map.

Argument

A subshift of finite type can be transformed into a finite graph. The graph's vertices are allowed words of length $k-1$, and edges indicate that they can be concatenated into an allowed word of length k . Thus legal sequences correspond to paths in the graph. The number of legal words of length n can be computed by powers of the adjacency matrix, turning a symbolic-dynamics problem into finite linear algebra.

The mod- N cat map is invertible if the matrix determinant is coprime to N , because the matrix is invertible over $\mathbb{Z}/N\mathbb{Z}$. Therefore the map is a permutation on a finite set. All orbits are eventually periodic, and periods can be computed.

Implementation Notes

"Chaos" on finite machines usually appears as long periods, sensitive dependence, and statistical complexity, rather than chaos in the infinite-precision sense. Documentation should state the state-space size and period-detection method.

Part 33: Discrete Optimal Control

Definition

Discrete optimal control studies state-action systems over finite time or infinite discounted time. A finite-time problem is:

$$s_{t+1} = T(s_t, a_t), J = \sum_{t=0}^{T-1} c(s_t, a_t) + h(s_T).$$

The goal is to choose a policy $\pi_t: S \rightarrow A$ that minimizes J . If states and actions are finite, dynamic programming can be used directly; if the state is an integer grid, one can approximate over a bounded region.

Argument

The Hamilton-Jacobi-Bellman equation in the discrete case is just the Bellman recurrence. Correctness is similar to Part 12, but here policies are emphasized. Define $\pi_t^*(s)$ as an action that minimizes the right-hand side of the Bellman equation. By backward induction in time, from any t, s , executing π^* yields cost $V_t(s)$, and any other policy has cost at least that.

If discounted infinite horizons are used, finite states and discount factor $\gamma < 1$ ensure that the Bellman operator is a contraction. Implementations usually use fixed-point or rational approximation iteration.

Implementation Notes

Robot navigation, game AI, inventory control, and task scheduling are often already discrete control problems. Continuous LQR and related methods can serve as background, but DCA documents should first give the state encoding, action set, integerized cost, and boundary conditions.

Part 34: Algebraic Coding and Cryptography

Definition

Algebraic coding uses linear spaces over finite fields. A linear code C is a subspace of F_q^n , represented by a generator matrix G or parity-check matrix H :

$$C = \{mG : m \in F_q^k\}, \quad c \in C \text{ if and only if } Hc^T = 0.$$

In cryptography, finite fields, elliptic curves, lattices, polynomial rings, and hash functions are all finite computable objects. Post-quantum lattice cryptography often operates over modular-integer polynomial rings.

Argument

Syndrome decoding of linear codes is based on the equality $H(c+e)^T = Hc^T + He^T = He^T$. Since the codeword term is 0, the syndrome depends only on the error vector. If a table from low-weight errors to syndromes is precomputed, errors can be corrected.

Correctness proofs for lattice cryptography usually show that after encryption and decryption, the result is $m + \text{noise}$, where the noise magnitude is below the rounding threshold, so rounding recovers m . Security relies on hardness assumptions such as LWE or Module-LWE, and cannot be derived directly from the DCA axioms of this document.

Implementation Notes

Cryptographic implementations must distinguish functional correctness, security proof, and side-channel-resistant implementation. DCA can help specify finite-ring operations and boundaries, but it cannot replace cryptanalysis. When referencing NIST standards, standard parameters and test vectors should be used.

Part 35: The Expressive Scope of DCA

Definition

The expressive scope of DCA can be summarized as follows: any object that can be represented by a finite encoding, manipulated by finite rules, and given a finite verification process is suitable for this framework. This includes finite combinatorics, graph theory, finite algebra, program semantics, finite probability, discrete geometry, and executable algorithms.

Objects that are not suitable for direct inclusion in the implementation layer include arbitrary nonconstructive real numbers, complete objects in infinite-dimensional spaces, nonconstructive existence depending on the axiom of choice, and continuous approximations without finite error bounds.

Argument

This scope is not an ontological conclusion but an engineering boundary. A computer can only store finitely many bits at any given moment, and any program execution can only go through finitely many steps. Therefore, if the goal is implementation, verification, or reproduction, finite encoding is a necessary condition. Conversely, as long as an object has a finite encoding and its operations have finite algorithms, it can be executed on some machine model.

The relationship with Peano arithmetic can be understood as follows: natural numbers and recursive definitions are core foundations, but DCA places greater emphasis on concrete encodings, word lengths, resource bounds, and program interfaces.

Implementation Notes

When writing open-source documents, it is advisable to avoid overly strong statements such as “some object does not exist.” A more accurate phrasing is: “the implementation layer of this document does not directly represent this object; if it is needed, it should enter the system through finite approximation, a generator, a specification, or a proof interface.”

Part 36: Physical Implementation Blueprint

Definition

A physical implementation blueprint is a layered sketch of an integer-computation system:

hardware layer: bits, registers, ALU, memory, interconnect; instruction layer: integer ISA and state-transition semantics; system layer: kernel, scheduling, memory isolation; library layer: finite algebra, matrices, NTT, graph algorithms; application layer: signal processing, optimization, AI inference, verification tools.

Argument

Layered correctness relies on interface contracts. Hardware satisfies instruction specifications; instruction specifications support compiler target semantics; the compiler preserves source-program semantics; library functions satisfy mathematical specifications; application programs call library functions while maintaining their own invariants. If every layer has clear preconditions and postconditions, they can be composed into an end-to-end correctness argument.

Performance analysis can also be discretized: area can be estimated by gate count or square micrometers, power by integer event counts, latency by cycles, and memory by bytes. All of these can enter an integer optimization model.

Implementation Notes

This is only an engineering blueprint; a complete hardware system need not be built at once. A more realistic route is to first build a reference interpreter, test suite, formalized fragments, and integer mathematics library, and then gradually replace lower layers with accelerators.

Part 37: Discrete Differential Topology and Morse Theory

Definition

Discrete Morse theory works on finite simplicial complexes or CW complexes. A discrete Morse function assigns a value to every simplex and constrains the number of neighboring simplices that “violate dimension monotonicity.” Unpaired simplices are called critical simplices.

A discrete gradient vector field consists of pairings between a simplex and one of its cofaces of one higher dimension, with the requirement that no closed gradient path exists. The number of critical simplices is related to the homological information of the space.

Argument

The core idea of discrete Morse theory is: if a low-dimensional simplex is paired with a high-dimensional simplex and such pairings do not form a closed loop, they can be treated as a locally removable structure. Each removal does not change the homotopy type. After repeated removals, the remaining critical simplices give a smaller model of the complex.

Morse inequalities state that the number m_k of critical simplices covers at least the Betti number β_k in weak form, i.e. $m_k \geq \beta_k$. Intuitively, every independent hole in homology must be carried by some critical structure.

Implementation Notes

Discrete Morse theory is well-suited for mesh simplification, topological denoising, and data-shape analysis. The implementation focus is maintaining simplex adjacency, pairing relations, and the no-closed-loop condition. All operations are finite combinatorial operations.

Part 38: Discrete Spectral Theory

Definition

Given a finite graph $G=(V,E)$, let A be the adjacency matrix, D the degree matrix, and the graph Laplacian:

$$L=D-A.$$

L is an integer matrix. Spectral theory often studies eigenvalues, but in DCA one may first study integer-computable invariants: kernel dimension, rank, connected-component count, quadratic form $x^T L x$, cut size, and boundary operators.

Argument

The key identity is:

$$x^T L x = \sum_{(u,v) \in E} (x_u - x_v)^2.$$

Expanding the right-hand side, each edge contributes $x_u^2 + x_v^2 - 2x_u x_v$; summing gives the degree term minus the adjacency term. Therefore $x^T L x \geq 0$, and if it is 0, then every edge has $x_u = x_v$. Thus vectors in $Lx=0$ are constant on each connected component, and the kernel dimension equals the number of connected components.

Implementation Notes

Spectral clustering often uses real-valued eigenvectors, but many graph problems can use integer alternatives: connected components by BFS/DFS, cut problems by maximum flow/minimum cut, and harmonic functions by solving rational linear equations. If eigenvalues are computed, floating-point approximation and error should be stated.

Part 39: Discrete Neural Architecture Search

Definition

Neural architecture search, or NAS, can be written as a finite optimization problem. Define an operation set O , layer-count upper bound L , and finite sets of channel choices, connection choices, and quantization bit-width choices. An architecture is a finite encoded string:

$$\text{arch} = (\text{op}_1, \text{ch}_1, \text{skip}_1, \dots, \text{op}_L, \text{ch}_L, \text{skip}_L).$$

Evaluation functions include accuracy, latency, parameter count, memory, and energy. If all measurements or estimates are integers, the result is a multi-objective integer optimization problem.

Argument

Because the search space is finite, an optimal architecture exists. If exhaustive search is used, correctness is immediate but cost is high. If random search, evolutionary algorithms, or Bayesian optimization is used, they are heuristics and cannot guarantee global optimality unless additional enumeration coverage or branch-and-bound proofs are attached.

Resource constraints can be written as predicates $R(\text{arch}) \leq B$. As long as construction, mutation, and filtering steps preserve or check this predicate, the output architecture is guaranteed to satisfy the budget.

Implementation Notes

NAS documentation should avoid reporting only “a better model was found.” It needs to preserve the search-space definition, random seed, evaluation protocol, hardware target, and constraints. DCA-style writing helps reproducibility: architecture encoding is a finite string, and the evaluation process is an executable program.

Part 40: DCA Bootstrap Interpreter

Definition

A bootstrap interpreter is an interpreter for the DCA language implemented in a language expressible by DCA. A program itself can be represented as a syntax tree or bytecode list. The interpreter state includes environment, stack, heap, and current instruction.

$\text{eval: Program} \times \text{Input} \times \text{Fuel} \rightarrow \text{Result}.$

Here Fuel is a step-count bound used to ensure that the interpreter function itself always terminates. If fuel is exhausted, it returns an unfinished state.

Argument

Interpreter correctness can be proved using small-step semantics. First define the abstract execution relation $P, s \rightarrow s'$ of the language. Then prove that every bytecode step executed by the interpreter simulates one abstract small step. By induction on the number of execution steps, if the abstract program obtains a result after k steps and $\text{Fuel} \geq k$, the interpreter obtains the same result.

Bootstrapping is not mysterious: as long as the language can represent its own syntax and can interpret those syntax objects, a self-interpreter can be written. Reflection comes from the fact that programs can be read and transformed as data.

Implementation Notes

Bootstrap systems can easily become complex. It is advisable to first implement a very small core language: integers, Booleans, conditionals, loops, function calls, and arrays. Type checking, compilation, and optimization can be added afterward. Every extension should have semantic rules and tests.

Part 41: Discrete Physical Mapping

Definition

Discrete physical mapping writes the computable part of a physical model as finite states or grid updates. Typical objects include lattice fields, particle systems, conserved quantities, local actions, and finite-difference equations. A discrete action can be written as:

$$S[\text{path}] = \sum_t L(x_t, x_{t+1}).$$

A discrete variational problem searches, within a finite path set, for a path that minimizes or makes stationary S .

Argument

The discrete Euler-Lagrange equation can be derived by finite variable perturbation. If an interior point x_t of a path changes, only the two neighboring terms $L(x_{t-1}, x_t)$ and $L(x_t, x_{t+1})$ are affected. Setting the first-order finite-difference variation to 0 yields a local equation. This derivation uses only finite sums and finite differences.

Discrete Noether-type conclusions can also be written as: if a local update rule is invariant under some finite transformation group, then a corresponding combinatorial quantity is preserved during the update. Concrete conserved quantities must be proved model by model.

Implementation Notes

Physical mapping is where overclaiming is easiest. This document only recommends treating discrete models as computational models or numerical models, not directly claiming they are the underlying structure of reality. If they are to be connected to experimental physics, scale, error, and testable predictions must be provided.

Part 42: Automated Theorem Proving

Definition

Automated theorem proving converts propositions into forms that a program can search. Propositional logic is often converted to CNF-SAT; equational logic can use rewrite systems; linear integer arithmetic can use Presburger algorithms or SMT; bit-vectors can be converted to SAT by bit-blasting.

Proof search returns two kinds of results: a satisfying assignment, or an unsatisfiability proof. Modern SAT solvers can often output checkable proof logs such as DRAT or LRAT.

Argument

Correctness of DPLL follows from completeness of branching. For a variable x , any satisfying assignment either sets $x=\text{true}$ or $x=\text{false}$. Therefore searching both branches does not miss a solution. Unit propagation preserves satisfiability equivalence: if all literals in a clause except one have become false, that remaining literal must be true; otherwise the clause is unsatisfied. Conflict backtracking excludes partial assignments already proved impossible.

If all branches conflict, the formula is unsatisfiable; if a complete conflict-free assignment is found, the formula is satisfiable.

Implementation Notes

Many finite propositions in DCA can be handed to SAT/SMT solvers. It is advisable to treat the prover as a backend and first compile high-level mathematical objects into Boolean, bit-vector, or integer constraints. For open-source trust, the solver version, input file, and checkable proof log should preferably be preserved.

Part 43: Fully Discrete Agent Concept

Definition

The “fully discrete agent” here is only an engineering concept: a system whose perception, memory, reasoning, planning, and action flow are all expressed using finite states, integer tensors, finite graphs, symbolic programs, and checkable constraints. It does not require a mysterious continuous model of mind, nor does it claim to achieve artificial general intelligence.

The agent state can be written as:

$\text{AgentState} = \text{Memory} \times \text{Belief} \times \text{Goal} \times \text{Policy} \times \text{RuntimeState}.$

Each component should have a finite encoding. The update rule is:

$\text{AgentState}_{[t+1]} = F(\text{AgentState}_t, \text{Observation}_t).$

Argument

The basic argument for safety and controllability comes from state-machine invariants. If a safety predicate $\text{Safe}(s)$ is defined, and one proves that the initial state is safe and every allowed action preserves safety:

$\text{Safe}(s) \text{ AND } \text{action_allowed}(s,a) \rightarrow \text{Safe}(\text{step}(s,a)),$

then by induction every finite execution prefix is safe. Planning correctness is similar: if the search algorithm only returns plans satisfying constraints, and if the executor only executes verified plans, then system behavior is restricted to the constraint set.

Implementation Notes

This direction should remain restrained. A more practical goal is to build auditable integer reasoning components, finite-state planners, verifiable safety constraints, and reproducible evaluations. If learning modules are involved, learned parameters, inference processes, and safety boundaries should be recorded separately.

References

1. Alan M. Turing, “On Computable Numbers, with an Application to the Entscheidungsproblem” , 1936. https://www.cs.virginia.edu/~robins/Turing_Paper_1936.pdf
2. Claude E. Shannon, “A Mathematical Theory of Communication” , Bell System Technical Journal, 1948. <https://people.math.harvard.edu/~ctm/home/text/others/shannon/entropy/entropy.pdf>
3. James W. Cooley and John W. Tukey, “An Algorithm for the Machine Calculation of Complex Fourier Series” , Mathematics of Computation, 1965. <https://web.stanford.edu/class/cme324/classics/cooley-tukey.pdf>
4. Mathieu Desbrun, Anil N. Hirani, Melvin Leok, Jerrold E. Marsden, “Discrete Exterior Calculus” , 2005. <https://arxiv.org/abs/math/0508341>
5. Keenan Crane, “Discrete Differential Geometry” course notes. <https://brickisland.net/ddg-web/>
6. Robin Forman, “A User’s Guide to Discrete Morse Theory” , 2002. <https://eudml.org/doc/123837>
7. Rocq Prover documentation, “Inductive types and recursive functions” . <https://rocq-prover.org/doc/V9.2.0/refman/language/core/inductive.html>

8. Benoit Jacob et al., “Quantization and Training of Neural Networks for Efficient Integer-Arithmetic-Only Inference” , CVPR 2018.
https://openaccess.thecvf.com/content_cvpr_2018/html/Jacob_Quantization_and_Training_CVPR_2018_paper.html
9. RISC-V Instruction Set Manual repository. <https://github.com/riscv/riscv-isa-manual>
10. NIST FIPS 203, Module-Lattice-Based Key-Encapsulation Mechanism Standard.
<https://csrc.nist.gov/pubs/fips/203/final>
11. Matthew Cook, “Universality in Elementary Cellular Automata” , Complex Systems, 2004.
https://www.complex-systems.com/abstracts/v15_i01_a01/
12. John von Neumann, “Theory of Self-Reproducing Automata” , 1966.
https://archive.org/details/theoryofselfrepr00vonn_0
13. Martin Davis, George Logemann, Donald Loveland, “A Machine Program for Theorem Proving” , Communications of the ACM, 1962. DOI: <https://doi.org/10.1145/368273.368557>
14. NIST, “FIPS 203: Module-Lattice-Based Key-Encapsulation Mechanism Standard” , 2024.
<https://csrc.nist.gov/pubs/fips/203/final>
15. NIST, “FIPS 204: Module-Lattice-Based Digital Signature Standard” , 2024. <https://csrc.nist.gov/pubs/fips/204/final>
16. NIST, “FIPS 205: Stateless Hash-Based Digital Signature Standard” , 2024. <https://csrc.nist.gov/pubs/fips/205/final>
17. Benoit Jacob et al., “Quantization and Training of Neural Networks for Efficient Integer-Arithmetic-Only Inference” , CVPR 2018.
https://openaccess.thecvf.com/content_cvpr_2018/html/Jacob_Quantization_and_Training_CVPR_2018_paper.html
18. LiteRT / TensorFlow Lite, “8-bit quantization specification” , 2024-2026 documentation.
https://developers.google.com/edge/litert/conversion/tensorflow/quantization/quantization_spec
19. ONNX Runtime, “Quantize ONNX models” .
<https://onnxruntime.ai/docs/performance/model-optimizations/quantization.html>
20. IREE, “Intermediate Representation Execution Environment” . <https://iree.dev/>
21. RISC-V International, “The RISC-V Instruction Set Manual” . <https://github.com/riscv/riscv-isa-manual>
22. Sail ISA specification language. <https://github.com/rem-s-project/sail>
23. Project Everest, verified software stack for secure communication. <https://project-everest.github.io/>
24. HAcl*, high-assurance cryptographic library. <https://hacl-star.github.io/>
25. Fiat-Crypto, formally verified cryptographic arithmetic generation. <https://github.com/mit-plv/fiat-crypto>
26. seL4 Foundation, verified microkernel project. <https://sel4.systems/>
27. CompCert, formally verified C compiler. <https://compcert.org/>
28. CakeML, verified implementation of ML. <https://cakeml.org/>
29. Rocq Prover documentation. <https://rocq-prover.org/doc/V9.2.0/refman/language/core/inductive.html>
30. Lean 4 / mathlib. <https://lean-lang.org/use-cases/mathlib/>
31. Isabelle theorem prover. <https://isabelle.in.tum.de/>
32. Dafny verification-aware programming language. <https://dafny.org/>
33. Kani Rust Verifier. <https://model-checking.github.io/kani/>
34. Z3 theorem prover. <https://github.com/Z3Prover/z3>
35. cvc5 SMT solver. <https://cvc5.github.io/>
36. Kissat SAT solver. <https://github.com/arminbiere/kissat>
37. CaDiCaL SAT solver. <https://github.com/arminbiere/cadical>

38. Google OR-Tools. <https://developers.google.com/optimization>
39. JuMP, Julia mathematical optimization modeling language. <https://jump.dev/JuMP.jl/stable/>
40. CVXPY, Python-embedded modeling language for convex optimization. <https://www.cvxpy.org/>
41. do-mpc, Python toolbox for model predictive control. <https://www.do-mpc.com/>
42. GUDHI, computational topology and topological data analysis library. <https://gudhi.inria.fr/index.html>
43. Ripser, persistent homology software. <https://github.com/Ripser/ripser>
44. Dionysus, computational topology package. <https://www.mrzv.org/software/dionysus/>
45. Guillaume Tauzin et al., “giotto-tda: A Topological Data Analysis Toolkit for Machine Learning and Data Exploration” , 2020. <https://arxiv.org/abs/2004.02551>
46. libigl, geometry processing library. <https://libigl.github.io/>
47. geometry-central, geometry processing library. <https://geometry-central.net/>
48. “Dxtr: A library for discrete exterior calculus” , Journal of Open Source Software, 2026. <https://www.theoj.org/joss-papers/joss.08110/10.21105.joss.08110.pdf>
49. SageMath, open-source mathematics software system. <https://www.sagemath.org/>
50. GAP, Groups, Algorithms, and Programming. <https://www.gap-system.org/>
51. FLINT, Fast Library for Number Theory. <https://flintlib.org/>
52. Macaulay2, software for algebraic geometry and commutative algebra. <https://macaulay2.com/>
53. Singular, computer algebra system for polynomial computations. <https://www.singular.uni-kl.de/>
54. arkworks algebra libraries. <https://github.com/arkworks-rs/algebra>
55. Winterfell STARK prover. <https://github.com/facebook/winterfell>
56. Plonky2 recursive SNARK/STARK library. <https://github.com/0xPolygonZero/plonky2>
57. AFF3CT, fast forward error correction toolbox. <https://aff3ct.github.io/>
58. NVIDIA Sionna, link-level and system-level simulations for communication systems. <https://developer.nvidia.com/sionna>
59. NetworkX, Python package for graph analysis. <https://networkx.org/>
60. SuiteSparse:GraphBLAS. <https://people.engr.tamu.edu/davis/GraphBLAS.html>
61. Qiskit SDK. <https://github.com/Qiskit/qiskit>
62. Cirq quantum computing framework. <https://quantumai.google/cirq>
63. QuTiP 5, Quantum Toolbox in Python. <https://arxiv.org/abs/2412.04705>
64. Stim stabilizer circuit simulator. <https://github.com/quantumlib/Stim>
65. Golly cellular automata explorer. <https://golly.sourceforge.io/>
66. TLA+ tools. <https://github.com/tlaplus/tlaplus>